Roteiro: Mural Anônimo do zero

Oct 2, 2026 · @Marcio

Como usar este roteiro

São 11 etapas curtas que constroem o mural anônimo do zero, cada uma em cima da anterior, e **toda etapa termina com um teste que precisa funcionar antes de seguir**. Se o teste falhar, o erro está no que vocês acabaram de escrever, o que deixa a depuração fácil.

Cada etapa traz quatro partes:

- **Por quê:** o conceito de redes que a etapa ensina.
- **O que escrever:** os arquivos e funções, descritos em palavras ou pseudocódigo. O código é de vocês.
- **Pronto quando:** o teste que prova que a etapa funciona.
- **Para conferir:** perguntas que cada um dos três deve saber responder antes de passar adiante.

Os trechos de código que aparecem são só chamadas de biblioteca que não dá para adivinhar, como as funções da `cryptography`. O zip que mandei antes serve de gabarito: escrevam primeiro e só consultem quando travarem de verdade. Comparar depois de tentar ensina mais do que copiar.

Visão geral

No fim vocês terão 5 programas que conversam só por sockets TCP:

```
                diretório (lista de relés + chaves públicas)
                  ↑ REGISTRAR            ↑ LISTAR_RELES
                relés R1..R4             cliente

cliente ──3 camadas──▶ R1 ──2 camadas──▶ R2 ──1 camada──▶ R3 ──pedido──▶
mural
        ◀──────────────── a resposta volta pelo mesmo caminho ────────────
```

Arquivo	Papel	Nasce na etapa
`comum.py`	enviar/receber mensagens e funções de criptografia	2 (cripto na 7 e 8)
`mural.py`	servidor que guarda os posts	4

Arquivo	Papel	Nasce na etapa
`cliente.py`	menu do usuário e montagem da cebola	5
`rele.py`	tira uma camada e repassa	6
`diretorio.py`	lista de relés e suas chaves	8

A ordem é proposital: primeiro o mural funciona **sem** anonimato (etapas 4–5), depois a mensagem passa por relés **em claro** (etapa 6) e só então entra a criptografia (etapas 7–8). Assim vocês veem no Wireshark o problema antes de ver a solução.

Divisão sugerida até domingo (troquem como quiserem; o importante é que todos entendam todas as partes):

Quando	Quem	O quê
Sexta à noite	os três juntos	Etapas 0 a 3: é a base que todo o resto usa
Sábado de manhã	Pietro	Etapa 4 (mural)
Sábado de manhã	Pedro Gabriel	Etapa 6, lado do relé
Sábado de manhã	Mateus	Etapa 5 (cliente) e Etapa 6, lado do cliente
Sábado à tarde	os três juntos	Etapas 7 e 8, a parte mais difícil
Domingo	divididos	Etapa 9, Etapa 10, relatório e ensaio, cada um explicando uma parte que **não** escreveu

Etapa 0 — Preparação (30 min)

Por quê: os três precisam do mesmo ambiente para que "na minha máquina funciona" não vire problema no domingo.

O que fazer:

1. Conferir `python3 --version` (3.8 ou mais novo).
2. Criar a pasta do projeto e um ambiente virtual: `python3 -m venv .venv` e `source .venv/bin/activate`.
3. `pip install cryptography` e salvar a dependência em `requirements.txt`.
4. Criar um repositório Git (pode ser privado no GitHub) para os três trabalharem em paralelo no sábado.
5. Instalar o Wireshark (`sudo apt install wireshark`) e testar uma captura na interface `lo`.

Pronto quando: `python3 -c "import cryptography; print('ok')"` imprime `ok`, e o Wireshark captura pacotes na `lo` quando vocês rodam `ping 127.0.0.1`.

Para conferir: o que é a interface de loopback (`lo`)? Por que, rodando tudo na mesma máquina, os pacotes aparecem nela e não na placa de rede?

Etapa 1 — Eco TCP (1 h)

Por quê: é o "olá, mundo" de redes, e todo o projeto é feito dessas mesmas chamadas. O servidor faz `socket → bind → listen → accept → recv/send → close`; o cliente faz `socket → connect → send/recv → close`.

O que escrever (arquivos de laboratório, que depois vão para o lixo):

- `eco_servidor.py`: escuta em `127.0.0.1:5000`, aceita uma conexão, recebe bytes e devolve o mesmo texto em maiúsculas.
- `eco_cliente.py`: lê uma frase do teclado, envia ao servidor e imprime a resposta.

Dica: o socket TCP é `socket.socket(socket.AF_INET, socket.SOCK_STREAM)`. No servidor, chame `setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)` antes do `bind`; sem isso, reiniciar o servidor dá "Address already in use" por alguns segundos.

Pronto quando: o cliente envia `oi rede` e recebe `OI REDE`. No Wireshark, com o filtro `tcp.port == 5000`, aparecem o SYN, SYN-ACK e ACK, depois os dados e por fim o FIN.

Para conferir:

- Em que linha do código acontece o handshake de 3 vias?
- Por que o servidor tem porta fixa e o cliente recebe uma porta diferente a cada execução? Onde essa porta aparece no Wireshark?
- O que o `accept()` devolve, e por que o servidor fica com dois sockets?

Etapa 2 — Mensagens com tamanho + JSON: `comum.py` (1 h 30)

Por quê: o TCP entrega um **fluxo de bytes**, não mensagens. Dois `send` seguidos podem chegar num único `recv`, e uma mensagem grande pode chegar em vários pedaços. Por isso precisamos de um protocolo próprio que diga onde cada mensagem termina (*framing*).

Experimento antes de escrever: no eco da etapa 1, faça o cliente chamar `send` duas vezes seguidas e o servidor um só `recv(4096)`, e veja as duas frases grudadas. Depois mande 200 KB e veja quantos bytes um único `recv` devolve.

O que escrever em `comum.py`:

- `enviar(sock, mensagem)`: transforma o dicionário em JSON, depois em bytes, e envia `[4 bytes com o tamanho][JSON]`. Use `sendall`, não `send`.
- `receber_bytes(sock, n)`: chama `recv` em laço até juntar exatamente `n` bytes; devolve `None` se a conexão fechar no meio.
- `receber(sock)`: lê os 4 bytes do tamanho, depois o JSON, e devolve o dicionário (ou `None`). Recuse tamanhos absurdos, por exemplo acima de 10 MB.
- `pedir("host:porta", mensagem)`: abre a conexão, envia, espera a resposta, fecha e devolve a resposta. Quase tudo no projeto vai usar essa função.

Dica: `struct.pack("!I", n)` gera os 4 bytes de um inteiro sem sinal em *network byte order* (big-endian), e `struct.unpack("!I", b)[0]` faz o caminho inverso. `socket.create_connection((host, porta), timeout=10)` abre a conexão do cliente numa linha.

Pronto quando: o eco, reescrito com `enviar` e `receber` trocando dicionários, devolve uma mensagem de 200 KB inteira (confiram o tamanho) e duas mensagens seguidas chegam separadas.

Para conferir:

- Por que `sendall` e não `send`?
- O que significa o `!` no `struct`, e por que os protocolos de rede usam big-endian?
- Qual o maior tamanho que 4 bytes conseguem anunciar?

Etapa 3 — Várias conexões ao mesmo tempo (45 min)

Por quê: um relé atende vários clientes ao mesmo tempo e, enquanto espera a resposta do próximo salto, não pode travar os outros. Usaremos uma thread por conexão, o modelo mais simples de servidor concorrente.

Experimento antes de escrever: coloque um `time.sleep(10)` no cliente do eco antes do envio. Com ele parado, abra um segundo cliente e veja que esse também fica esperando: o servidor só volta ao `accept` quando termina o primeiro.

O que escrever:

- Em `comum.py`, a função `criar_servidor(host, porta)` com `socket → setsockopt → bind → listen`, porque todos os servidores vão repetir isso.
- No servidor de eco, a função `atender(conexao, endereco)` com o trabalho de uma conexão. O laço principal só faz `accept` e dispara uma thread: `threading.Thread(target=atender, args=(conexao, endereco), daemon=True).start()`.

Pronto quando: com o cliente lento parado no `sleep`, um segundo cliente recebe a resposta na hora.

Para conferir:

- O que as threads compartilham entre si? O que aconteceria se duas threads acrescentassem um item à mesma lista exatamente ao mesmo tempo? Isso volta na etapa 4.
- O que `daemon=True` muda quando apertamos Ctrl+C?
- Qual seria a alternativa a threads, por exemplo `select` ou `asyncio`, e por que escolhemos threads?

Etapa 4 — Servidor do mural, ainda sem anonimato (2 h)

Por quê: aqui vocês projetam o protocolo da aplicação: quais mensagens existem (sintaxe), o que o servidor faz com cada uma (semântica) e o que responde quando algo dá errado. A tabela abaixo vai direto para a seção 3 do relatório.

Pedido	Campos	Resposta
`TOPICOS`	—	`TOPICOS` com a lista de tópicos
`LISTAR`	topico	`POSTS` com id, título, autor e nº de respostas
`LER`	id	`POST` com texto e respostas
`POSTAR`	topico, titulo, texto, apelido	`OK` com o id do post novo
`RESPONDER`	id, texto, apelido	`OK`
qualquer erro	—	`ERRO` com um `motivo`

O que escrever em `mural.py`:

- Uma lista global `posts`, uma lista fixa de tópicos e uma trava `threading.Lock()`. Toda leitura ou escrita em `posts` acontece dentro de `with trava:`.
- `tratar_pedido(pedido)`: recebe o dicionário e devolve o dicionário de resposta. Não usa sockets, então dá para testar sem rede.
- `atender(conexao, endereco)`: `receber`, imprime o tipo do pedido e de qual IP:porta veio, chama `tratar_pedido`, `enviar` e fecha. Uma conexão para cada pedido.
- `main()`: `criar_servidor` na porta 7000 e o laço de `accept` com threads, como na etapa 3.

Pronto quando: num `python3` interativo, usando `pedir("127.0.0.1:7000", {...})`, vocês conseguem postar, listar, ler e responder; um tópico que não existe e um id que não existe devolvem `ERRO`.

Para conferir:

- Por que a trava é necessária se cada pedido roda em uma thread?
- Por que separar `tratar_pedido` de `atender`?
- Que IP e porta o mural imprime para cada pedido? Guardem essa resposta: depois da etapa 6 ela muda.

Etapa 5 — Cliente com menu, falando direto com o mural (1 h 30)

Por quê: ter a aplicação inteira funcionando antes de mexer no anonimato. A decisão importante desta etapa é que **todo pedido passa por uma única função**, `enviar_ao_mural(pedido)`. Hoje ela só chama `pedir(MURAL, pedido)`; nas etapas 6 e 8 só ela muda, e o menu continua igual. Isso é arquitetura em camadas: a aplicação não sabe como a mensagem viaja.

O que escrever em `cliente.py`:

- Constante `MURAL = "127.0.0.1:7000"` e a função `enviar_ao_mural(pedido)`.
- `main()`: pergunta o apelido uma vez e mostra um menu numerado em laço: ver tópicos, listar um tópico, ler um post, postar, responder e sair. Cada opção monta o dicionário do pedido, chama `enviar_ao_mural` e imprime a resposta de um jeito legível.
- Trate `ValueError` (id digitado errado) e `OSError` (servidor fora do ar), para o menu não morrer.

Pronto quando: com dois terminais de cliente abertos, um posta e o outro lista e lê o post. Desligar o mural com o menu aberto mostra uma mensagem de erro em vez de travar o programa.

Para conferir:

- Quantas conexões TCP cada opção do menu abre? Confiram no Wireshark.
- Se amanhã trocássemos TCP por outra coisa, quais funções do cliente mudariam?

Etapa 6 — Três relés sem criptografia (2 h)

Por quê: é aqui que nasce o roteamento em nível de aplicação. Cada relé recebe um pacote que diz "repasse isto para tal endereço" e não precisa saber de mais nada. Fazer primeiro **sem** cifra permite ver no Wireshark exatamente o problema que a etapa 8 resolve.

Formato da camada (por enquanto em claro): `{"tipo": "CEBOLA", "proximo": "host:porta", "conteudo": {...}}`. O `conteudo` é outra camada ou, na última, o pedido ao mural.

O que escrever:

- `rele.py`, executado como `python3 rele.py R1 9001`. A função `atender` recebe o pacote, imprime "recebi de X, repassando para Y", chama `pedir(proximo, conteudo)`, devolve a resposta a quem chamou e fecha. Se o próximo não responder, devolve `ERRO`.
- No cliente, uma lista fixa com os 3 endereços dos relés e a função `montar_camadas(pedido, caminho)`. A cebola é montada **de dentro para fora**:

```
conteudo ← pedido
proximo  ← endereço do MURAL
para cada relé do caminho, do ÚLTIMO para o PRIMEIRO:
    conteudo ← {tipo: CEBOLA, proximo: proximo, conteudo: conteudo}
    proximo  ← endereço desse relé
envie conteudo para o PRIMEIRO relé do caminho
```

- `enviar_ao_mural` passa a chamar `montar_camadas` e enviar ao primeiro relé. O menu não muda.

Pronto quando: o menu funciona como antes, cada relé imprime uma linha por pedido e o mural passa a imprimir a porta do relé 3 em vez da do cliente.

Agora olhem o Wireshark: em *Follow TCP Stream* entre o relé 1 e o relé 2, o texto do post está legível, e o relé 1 consegue ler o caminho inteiro dentro do pacote. Ou seja, **ainda não há anonimato**.

Para conferir:

- Por que a cebola precisa ser montada de dentro para fora?
- O que o relé 1 sabe nesta versão, e o que ele deveria saber?
- Quantas mensagens um pedido gera agora, somando ida e volta?

Etapa 7 — Criptografia simétrica com Fernet (1 h)

Por quê: para esconder o conteúdo de cada camada precisamos cifrar. O Fernet, da biblioteca `cryptography`, é cifra simétrica (AES): a **mesma chave** cifra e decifra, e ele ainda detecta se o texto foi alterado.

Experimento no `python3` interativo:

```
from cryptography.fernet import Fernet
chave = Fernet.generate_key()
token = Fernet(chave).encrypt(b"ola rede")
Fernet(chave).decrypt(token)
```

Cifrem a mesma frase duas vezes e comparem os resultados; comparem o tamanho do token com o da frase; tentem decifrar com outra chave.

O que escrever em `comum.py`: `nova_chave()`; `cifrar(chave, objeto)`, que transforma o dicionário em JSON, cifra e devolve texto (para caber no JSON da mensagem); e `decifrar(chave, texto)`, que faz o caminho inverso.

Pronto quando: `decifrar(k, cifrar(k, {"a": 1}))` devolve `{"a": 1}`, e decifrar com a chave errada levanta `InvalidToken`.

O problema que fica para a etapa 8: como o cliente combina uma chave com cada relé sem que mais ninguém a conheça? Uma chave fixa por relé, escrita num arquivo, seria compartilhada por todos os usuários daquele relé.

Para conferir:

- Por que cifrar a mesma mensagem duas vezes dá resultados diferentes?
- Por que o token fica maior que o texto original?
- Qual a diferença entre cifra simétrica e assimétrica?

Etapa 8 — RSA e diretório: a cebola de verdade (3 h, os três juntos)

Por quê: com RSA (cifra assimétrica) cada relé publica uma chave pública; qualquer um cifra com ela, mas só o relé, com a privada, decifra. O RSA é lento e só cifra poucos bytes, por isso usamos **cifra híbrida**: o cliente sorteia uma chave Fernet para cada relé e manda essa chave cifrada com o RSA do relé.

8a — RSA no `python3` interativo:

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes, serialization

privada = rsa.generate_private_key(public_exponent=65537, key_size=2048)
pem = privada.public_key().public_bytes(serialization.Encoding.PEM,
          serialization.PublicFormat.SubjectPublicKeyInfo).decode()
oaep = padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)

publica = serialization.load_pem_public_key(pem.encode())
cifrado = publica.encrypt(b"uma chave fernet", oaep)
privada.decrypt(cifrado, oaep)
```

Em `comum.py`: `gerar_chaves()`, que devolve a privada e o PEM da pública; `cifrar_rsa(pem, dados)`, que devolve texto em hexadecimal; e `decifrar_rsa(privada, texto_hex)`.

8b — `diretorio.py` (porta 9000): `REGISTRAR` {nome, endereco, chave_publica} guarda o relé e responde `OK`; `LISTAR_RELES` responde `RELES` com a lista. Cada relé, ao ligar, gera seu par de chaves e se registra. No cliente, `escolher_caminho()` pede a lista e sorteia 3 relés com `random.sample`.

8c — a camada passa a ser cifrada:

```
{tipo: CEBOLA,
 chave: RSA_do_relé( chave Fernet sorteada ),
 dados: Fernet( {proximo, conteudo} )}
```

- **Cliente**, em `montar_cebola`: o mesmo laço da etapa 6, mas cada camada sorteia uma chave Fernet nova, cifra `{proximo, conteudo}` com ela e cifra a chave com o RSA daquele relé. Guarde as 3 chaves na ordem R1, R2, R3.
- **Relé**, em `atender`: abre `chave` com sua privada, abre `dados` com essa chave, repassa `conteudo` para `proximo` e devolve `{tipo: RESPOSTA, dados: Fernet(resposta)}` cifrado com a mesma chave.
- **Cliente**, em `abrir_resposta`: a resposta chega com 3 camadas, a do R1 por fora. Decifre com a chave do R1, depois a do R2, depois a do R3.

Pronto quando: o menu funciona igual; no Wireshark, nenhum trecho entre relés tem texto legível, e cada salto carrega um pacote menor que o anterior. Só o trecho R3 → mural mostra o pedido em claro.

Para conferir:

- Por que não cifrar tudo direto com RSA?

- Por que a resposta é decifrada na ordem R1, R2, R3, e a cebola montada na ordem R3, R2, R1?
- O que cada relé sabe agora? E o mural?
- Por que sortear chaves Fernet novas a cada pedido, em vez de reaproveitar?

Etapa 9 — Senha de autor e APAGAR (1 h)

Por quê: num mural anônimo não há login, mas o autor precisa provar que um post é dele, por exemplo para apagá-lo. A solução usa uma função de hash: o servidor guarda só o hash da senha e mostra parte dele ao lado do apelido.

O que escrever:

- No cliente: pedir a senha de autor uma vez (pode ficar vazia), mandar `senha` junto com `POSTAR` e `RESPONDER`, e uma opção nova no menu, "apagar post", que manda `APAGAR` {id, senha}.
- No mural: se veio senha, calcular `hashlib.sha256(senha.encode()).hexdigest()`, guardar o hash no post e exibir o autor como `apelido !` mais os 8 primeiros caracteres do hash. `APAGAR` só funciona se o hash da senha recebida for igual ao guardado. Nunca guarde a senha em si.

Pronto quando: o mesmo apelido com senhas diferentes aparece com códigos diferentes, `APAGAR` com a senha errada devolve `ERRO` e com a certa o post some da lista.

Para conferir:

- Por que guardar o hash, e não a senha?
- Alguém consegue usar o apelido "ana" de outra pessoa? E o código depois do `!`?
- O relé de saída vê a senha em claro no pedido. Isso é um problema? Como poderia ser evitado? Vale citar como limitação no relatório.

Etapa 10 — Teste automático, Wireshark e números do relatório (domingo)

Por quê: a seção 6 do relatório pede testes e resultados, e na apresentação nada pode depender de digitar rápido.

O que escrever:

1. `iniciar_rede.sh`: liga o diretório, 4 relés e o mural em segundo plano (`python3 -u programa.py ... > logs/nome.txt &`) e guarda os PIDs num arquivo. `parar_rede.sh` faz o `kill` desses PIDs. Com 4 relés, o caminho sorteado muda de uma execução para outra.

2. `teste.py`: sobe a rede em outras portas com `subprocess`, chama direto as funções do cliente (`escolher_caminho`, `montar_cebola`, `abrir_resposta`) e confere cada resultado: postar, listar, ler, responder, apagar com senha errada e certa, tópico inexistente e outro caminho. No fim, conte nos logs dos relés as linhas "repassando": devem ser 3 por pedido.
3. Medições para o relatório:
   - o tamanho do mesmo `POSTAR` em cada trecho (dica: `len(json.dumps(...))` de `montar_cebola` com o caminho inteiro, sem o primeiro relé e só com o último);
   - o tempo médio de 20 pedidos direto ao mural e de 20 pelo circuito.
4. Prints do Wireshark: o handshake da etapa 1; o texto legível entre relés da etapa 6 (se guardarem uma cópia dessa versão); o mesmo trecho cifrado depois da etapa 8; e o pedido em claro entre o relé 3 e o mural.

Pronto quando: `python3 teste.py` termina com todas as verificações OK, e vocês têm as medições e os prints.

Último passo: o relatório que mandei descreve esta mesma arquitetura, mas os números dele vieram da minha versão. Troquem pelos que vocês mediram e revisem o texto para descrever exatamente o código de vocês.

Perguntas que o professor pode fazer

Treinem respondendo em voz alta, cada um respondendo sobre uma parte que **não** escreveu. A segunda coluna diz o que a resposta precisa conter, não a resposta pronta.

Pergunta	A resposta precisa ter
Por que TCP e não UDP?	pacotes grandes que precisam chegar inteiros e em ordem para serem decifrados; o que teriam que reimplementar com UDP
Como o receptor sabe onde termina uma mensagem?	TCP é fluxo de bytes; o prefixo de 4 bytes com o tamanho; `receber_bytes` em laço
Qual é a arquitetura: CS, P2P ou híbrida?	o diretório e o mural são CS; os relés são P2P (servidor e cliente ao mesmo tempo); juntos, híbrida
O que o relé 2 sabe sobre um pedido?	só quem entregou (R1) e para quem repassar (R3); não sabe o conteúdo, o cliente nem o destino final
Por que montar a cebola de dentro para fora?	a camada de fora precisa conter a de dentro já cifrada

Pergunta	A resposta precisa ter
Por que RSA e Fernet, e não só um dos dois?	RSA resolve a troca de chave mas é lento e só cifra poucos bytes; Fernet é rápido mas precisa de chave combinada
Mostre no Wireshark onde está o anonimato	trechos entre relés ilegíveis; o mural recebe conexões só do último relé
Quantas mensagens um pedido gera?	4 na ida e 4 na volta, uma conexão TCP por salto
Quais são as limitações?	tamanho diferente em cada salto permite rastrear; a saída lê o pedido e a senha; dados só na memória; diretório não percebe relé fora do ar
O que mudaria para ficar mais parecido com o Tor?	células de tamanho fixo, circuitos que duram vários pedidos, troca de chaves Diffie-Hellman